

AI Audit (Hash Chain) — Admin User Guide

Audience: Fynla administrators with access to the Admin panel. **What this is:** A tamper-evident ledger of every tool call Fyn (the AI) makes on behalf of users — every record it writes, every recommendation it generates, every advice-mode write attempt that was stripped before the model ever saw it.

Why we have it

Before this feature, AI tool calls were recorded as text lines in `storage/logs/laravel.log` (`[AI-AUDIT] tool=create_savings_account user=42 ...`). Anyone with file write access on the server could edit, delete, or fabricate those lines and you would never know.

For an app where Fyn writes to the user's financial records on their behalf, that is not good enough. The hash-chain audit gives three guarantees:

1. **Every AI tool call is recorded** as a row in the `ai_audit_events` table — tool name, operation, status, user, conversation, the entity that was touched (e.g. `SavingsAccount#42`), an input/result summary, and a timestamp.
2. **Tamper detection.** Each row carries a SHA-256 fingerprint (`row_hash`) computed from the previous row's hash chained with its own payload. Mutate any historical row and its `row_hash` no longer matches — every subsequent row also fails to re-derive, and the chain verifier reports `chain_valid: false, broken_at: <id>`. Someone with raw database write access cannot quietly "fix" history.
3. **Forgery resistance.** Each row also carries an HMAC signature keyed on a secret (`AI_AUDIT_HMAC_KEY`) held only on the server. Even if an attacker dumped the table and tried to insert a new "valid" tail row, they don't have the key, so the signature won't verify.

This implements the spec invariants **INV-2.10.2** (hash-chain audit) and **INV-2.5.4** (audit trail must match reality). Browser scenario **BS-15** tests it end-to-end.

How to open the view

1. Sign in as an admin (e.g. `chris@fynla.org`).
 2. Navigate to **Admin** in the side nav.
 3. Open the **AI** tab and click the **AI Audit** sub-menu.
 4. The panel opens on the **Conversations** tab. Click the **Chain view** sub-tab to switch to the hash-chain ledger.
-

The two tabs

Tab 1 — Conversations

A three-pane drilldown for reading what Fyn actually said and did:

- **Left pane:** every user with at least one AI conversation, searchable by name or email. Preview personas are flagged with a violet "Preview" badge.
- **Middle pane:** that user's conversations, newest first, with token counts and message counts.
- **Right pane:** the full message thread — user prompts, Fyn's replies, token usage, model used, and three expandable sections per assistant message:
 - **Tool Calls** — every tool Fyn invoked in that turn, with a one-line result summary.
 - **System Prompt** — the exact system prompt the model saw for that turn (with an approximate token count).
 - **Validation Violations** — guardrail rules that fired (e.g. core-identity tone, prompt-injection sanitisation), shown as raspberry pills.
- **Advice log header:** above the messages, the conversation's classified `query_type`, `related` topics, and a KYC pass/block badge if the conversation hit the KYC gate.

Use this tab for content review — "what did Fyn say to user X and why".

Tab 2 — Chain view

The forensic ledger. This is the S0.12 surface.

Integrity banner

At the top right of the panel, one of three states renders:

Banner	Meaning
Chain valid · 1,234 rows · tip a3f2c9d1b4e7... (spring/green)	Every row re-hashed cleanly back to its stored <code>row_hash</code> . The <code>tip_hash</code> is the most recent row's hash — quote this if anyone ever asks "prove the audit log hasn't been touched since date X". The full 64-character hash is exposed in the <code>data-tip-hash</code> attribute and the hover tooltip.
Chain broken at row #4217 (raspberry/red)	Re-hashing failed at that row id. Means a row was mutated, deleted, or the HMAC key changed. Investigate immediately.
Verifying chain... (neutral)	Verification in progress. Re-rendered every time the tab is opened or Re-verify is clicked.

The **Re-verify** button forces a fresh end-to-end walk on demand. The chain is also walked automatically every Sunday at 04:30 by `php artisan ai:audit:verify-chain` (scheduled in `app/Console/Kernel.php`).

Filters

Three filters above the table, all combinable:

- **Operation:** `read`, `write`, `handoff`, `classify`.
- **Status:** `dispatched`, `persisted`, `failed`, `stripped`.
- **User ID:** numeric — the internal `users.id`. Use the Conversations tab to look up an id if you only have an email.

The right-hand counter shows the total number of rows matching the current filter set. Pagination at the bottom is 50 rows per page.

Table columns

Column	What it tells you
#	Row id. Ascending = chronological order (although the table is rendered descending by default — newest first).
Tool	The tool Fyn invoked, e.g. <code>create_savings_account</code> , <code>update_property</code> , <code>delegate_to_capture</code> , <code>get_recommendations</code> .
Op	<code>read</code> (look-ups), <code>write</code> (records persisted on the user's behalf — the consequential ones), <code>handoff</code> (advice-mode → capture-mode switch when the user asks Fyn to record something), <code>classify</code> (QueryClassifier audits — Sprint 1+).
Status	<code>dispatched</code> (entered the tool), <code>persisted</code> (success — record written), <code>failed</code> (caught exception), <code>stripped</code> (an advice-mode write tool that was removed from the catalogue before the LLM saw it — proves Advice Fyn's read-only contract held).
User	The internal user id.
Entity	What got written, e.g. <code>SavingsAccount#42</code> . Only set on <code>persisted</code> rows.
Hash	First 12 characters of <code>row_hash</code> (mono-spaced) — the cryptographic fingerprint of this row + its lineage.
When	<code>signed_at</code> — the timestamp at the moment of append.
Status pill colours	<code>persisted</code> = spring/green, <code>failed</code> = raspberry/red, <code>stripped</code> = violet, <code>dispatched</code> = horizon/blue.

Status badge interpretation

The four statuses tell you the lifecycle of a tool call:

- **dispatched** — Fyn started executing the tool. Always written first.
- **persisted** — the tool completed successfully and a record was written to the user's data. Look for this immediately after a `dispatched` row with the same `tool_name` to confirm the dispatch landed.
- **failed** — the tool threw an exception. The exception is summarised in `result_summary`. Use this with **Op = write** to find broken writes.
- **stripped** — the tool was a write tool, but the user was in advice mode (post-onboarding). The tool was stripped from the catalogue before the LLM ever saw it. This is the audit proof that Advice Fyn's read-only contract held — see the canonical Two-Fyn contract in [April/April24Updates/spec/00-canonical.md](#).

Common admin tasks

"Is the audit log intact right now?"

Open the Chain view tab. Read the banner. Click **Re-verify** if you want a fresh walk.

"Show me every write Fyn made for user 42"

Filter **User ID = 42**, **Op = write**. Sort is descending — newest first.

"Did this dispatch ever land?"

Find the **dispatched** row. Look for the **persisted** or **failed** row immediately after it (same **tool_name**, same **user_id**, sequential **id**).

"Were any writes stripped from advice mode?"

Filter **Status = stripped**. Every row here is a write attempt that Advice Fyn correctly refused to expose to the LLM. If this list ever contains rows for users who are still mid-onboarding, the persona-split has regressed and the canonical contract is broken.

"What did Fyn actually do during this conversation?"

Cross-reference the **Conversations** tab (content — what was said) with the **Chain** tab filtered by **user_id** (consequences — what was written, when). The **conversation_id** column is not displayed in the table, but it's present on every row and is filterable via the API if needed.

"Prove to a regulator the log hasn't been touched since date X"

Quote the **tip_hash** from the banner at date X (it's in the **data-tip-hash** attribute on the banner element if you take a screenshot). Any change to any row at or before date X will change the current tip hash. The HMAC signature on each row prevents anyone from swapping in a forged tip without the key.

"I need a forensic deep-dive — full hashes, signatures, JSON payloads"

The admin UI is a triage view. The full row (all 64 chars of **prev_hash**, **row_hash**, **signature**, the full **input_summary** and **result_summary** JSON) is returned by the API at **GET /api/admin/ai-audit/chain** and **GET /api/admin/ai-audit/chain/verify**. For ops, run:

```
php artisan ai:audit:verify-chain
```

It returns **{"chain_valid": true, "tip_hash": "...", "row_count": N}** (exit 0) or **{"chain_valid": false, "broken_at": <id>, "row_count": N}** (exit 1).

What the view deliberately does NOT show

To keep the view a triage surface rather than a forensic dump:

- The full **prev_hash** (only displayed via API).
- The full 64-character **row_hash** (only the 12-character prefix is shown; full hash is in the API response).
- The HMAC **signature** (API only).

- The full `input_summary` / `result_summary` JSON (API only).
- The `signed_at` ISO precision beyond the formatted timestamp.

If you need any of these, hit the API directly or query the `ai_audit_events` table.

Retention

Old rows are pruned weekly (Sunday 04:00) by `AiAuditRetentionJob`:

- **7-year retention:** rows where `operation = 'write'` OR `tool_name = 'get_recommendations'`. These are the regulator-relevant trail of writes and personalised recommendations.
- **2-year retention:** all other rows (read tools, classifier hits, handoff chrome, stripped events).

Important: the job **deletes** aged-out rows — it does not pseudonymise them in place. Mutating any historical row would break the hash chain and `verifyChain()` would correctly report a break. If GDPR-style pseudonymisation of older rows is needed for an export, it must be done in a separate read-only export view that re-serialises the row with PII fields swapped — the source rows stay untouched and the chain stays verifiable.

After pruning, the chain remains coherent for the retained tail: the new "first" row simply has no predecessor, and `verifyChain()` walks from the earliest surviving id with the canonical zero-hash predecessor.

Troubleshooting

Symptom	What to check
Banner says Chain broken at row #N	Pull the row at <code>id = N</code> directly from the database. Compare its <code>row_hash</code> with the verifier's expected hash (re-run <code>php artisan ai:audit:verify-chain</code> for the broken_at id). Most likely causes: someone with raw DB write access mutated a row; the <code>AI_AUDIT_HMAC_KEY</code> env var changed; a migration accidentally truncated and re-inserted rows.
Chain view shows "No audit rows match these filters" but you know writes happened	Clear all three filters (operation, status, user_id) and re-load. The filters are AND-combined.
Re-verify spins forever	The verifier walks the entire table in id order. On a large chain (>100k rows) this can take seconds. If it never resolves, check <code>storage/logs/laravel.log</code> for the chain endpoint timing out.

Symptom	What to check
New tool calls aren't appearing	Confirm the tool routing through <code>CoordinatingAgent::executeTool</code> actually calls <code>AuditChainService::append()</code> . Tools that bypass the agent (direct service calls) won't be audited — that is by design, the audit covers AI-mediated writes only.
Conversations tab shows users but Chain tab shows zero rows	The two tabs read different tables (<code>ai_messages</code> vs <code>ai_audit_events</code>). Older conversations from before S0.12 went live won't have audit rows. Check the migration date: <code>2026_04_25_000013_create_ai_audit_events_table.php</code> .

Related references

- **Spec:** `April/April24Updates/spec/01-invariants.md` — INV-2.10.2 (hash-chain audit) and INV-2.5.4 (audit trail matches reality).
- **Browser scenario:** `tests/Browser/scenarios/BS-15-hash-chain-audit-admin-view.php` — the end-to-end acceptance test for this admin view.
- **Pest siblings:** `tests/Feature/Audit/HashChainTest.php`, `tests/Feature/Audit/ChainTamperDetectionTest.php`, `tests/Feature/Audit/RetentionPseudonymisationTest.php`.
- **Two-Fyn contract:** `April/April24Updates/spec/00-canonical.md` — explains why `stripped` rows exist (Advice Fyn is read-only).
- **Technical doc:** `docs/manuals/ai-audit-hash-chain-technical.md` — schema, hashing algorithm, API, retention internals.